Helpdesk Ticket Management System

Autonomous AI Agent & Gemini Classification Verification

This document contains representative test cases and seeded data designed to validate the system's duplicate ticket mitigation logic, priority assessment, and automated classification routing workflows.

Document Type	Sample Data & Verification Guide
Target Component	AI Classification and Duplicate Check Pipeline
Evaluation Framework	FastAPI REST & FastMCP stdio Protocols
Generation Date	June 10, 2026

1. Purpose & Directory Structure

The `sample_data/` directory houses inputs and outputs for regression testing. These test cases feed directly into standard testing suites (e.g., `test_agent.py`) to guarantee that issue classification and similarity matching logic remain consistent across core upgrades.

File Path	Description
sample_data/README.md	Documentation of test scenarios and commands
sample_data/input_issue_1.txt	Raw ticket text for Case 1 (payments down)
sample_data/expected_output_1.json	Expected classification properties for Case 1
sample_data/input_issue_2.txt	Raw ticket text for Case 2 (slow queries)
sample_data/expected_output_2.json	Expected comment action properties for Case 2

2. Test Cases Specification

Case 1: Critical Payment System Outage (New Ticket)

A severe issue indicating business-critical systems are offline. Since no duplicate ticket exists in the database, the ReAct loop automatically classifies this as CRITICAL urgency under the 'Payments' category and creates a new ticket.

input_issue_1.txt

```
Production payment system is down
```

expected_output_1.json

```
{
  "priority": "CRITICAL",
  "category": "Payments",
  "action": "Create Ticket"
}
```

Case 2: Query Timeout Warning (Duplicate Verification)

An issue describing query latency overlaps with active tickets in the seeded database. Jaccard similarity token checking triggers duplicate mitigation (>= 25% threshold), routing the agent to append a comment to the existing ticket rather than duplicating it.

input_issue_2.txt

```
FastAPI is slow and throwing query timeouts.
```

expected_output_2.json

```
{
  "priority": "HIGH",
  "category": "Software",
  "action": "Comment on Ticket"
}
```

3. Verification & Execution Instructions

The classification parsing engine and ReAct workspace triggers are tested systematically by reading these files in isolated runner environments. Use the following commands to execute tests locally or inspect agent classification fallbacks.

Command Console

```
# 1. Run the test agent runner framework
python project/agent/test_agent.py

# 2. Run the Gemini mock and API classifier test suite
python project/agent/test_gemini_classifier.py
```

The automated classification will leverage your configured `GEMINI_API_KEY` in your local `.env` environment. If the API key is not present or rate-limits are hit, the workflow falls back gracefully to local rule-based parsing.